Southern New Hampshire University

CS 350 Final Project Thermostat Lab Guide

It is important to note the distinct differences in the cyber-physical actions of a thermostat. The actions of the thermostat depend on whether it is in heating mode, cooling mode, or adaptive mode, where it is trying to maintain a specific temperature.

In heating mode, when you select a button to increase or decrease the room's temperature, the thermostat establishes a target temperature that is referred to as its set point. When the thermostat compares the temperature read from the sensor to the established set point, a decision is made. If the temperature is below the set point, it turns the heater on. However, if the temperature is at or above the set point, it turns the heater off.

In cooling mode, when you select a button to increase or decrease the room's temperature, the thermostat establishes a target temperature that is referred to as its set point. When the thermostat compares the temperature read from the sensor to the established set point, a decision is made. If the temperature is above the set point, it turns the air conditioner on. However, if the temperature is at or below the set point, it turns the air conditioner off.

In adaptive mode, when you select a button to increase or decrease the temperature in the room, the thermostat establishes a target temperature, referred to as its set point. When the thermostat compares the temperature read from the sensor to the established set point, a decision is made. If the temperature is above the set point, it turns the air conditioner on. However, if the temperature is at the set point, it turns the air conditioner (or the heater) off. Similarly, the heater is turned on if the temperature is below the set point.

For this project, you will only consider heating or cooling, not adaptive temperature controls.

Because you are not adding a new peripheral to the circuit but two additional buttons instead, you will not need to add any more software packages to your existing stack.

Step 1: Power off your Raspberry Pi

Your Raspberry Pi must be shut down and unplugged to continue. To safely shut down the operating system running on your Raspberry Pi, enter the following command from a terminal (or the console) of the Raspberry Pi:

```
sudo halt
```

You will have to enter your password as you are executing the command with root privileges by calling it with *sudo*. Please wait at least two minutes after your terminal window closes before **unplugging** your Raspberry Pi.

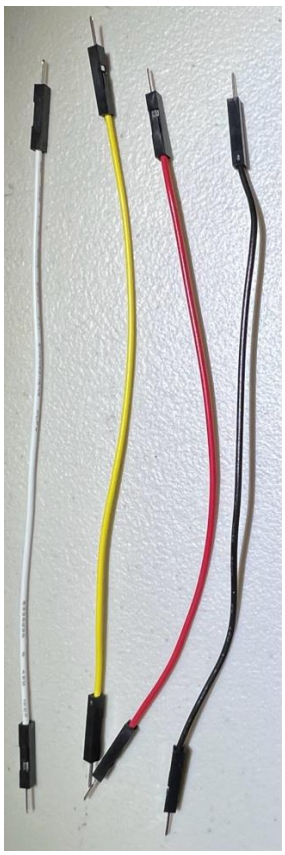
Step 2: Completing the Final Circuit

Components

- 2 x buttons (with your choice of keycap)
- 2 x 10K ohm resistor (stripe pattern: brown, black, orange)
- 4 x male-to-male jumper wires

Button

10K ohm resistor



Male-to-male jumper wires

The first component you will be connecting is a button. The button takes up a lot of real estate on the solderless breadboard, so you need to make certain that you position it so that you can make the appropriate connections. Each button has four pins, with the pair of pins opposite each other connected. Pressing the button makes momentary contact between the normally separated pins.

Note: Through this point, you have been using the numbering on the right-hand side of the solderless breadboard (looking at the board where the GPIO breakout board looks like a capital T). However, for the buttons, you will switch to the numbering on the left-hand side of the board as it is closer to the components. Position the button so that the pins on one side of the switch are in Column D, Rows 27 and 29, while the other is connected to Column G, Rows 27 and 29.

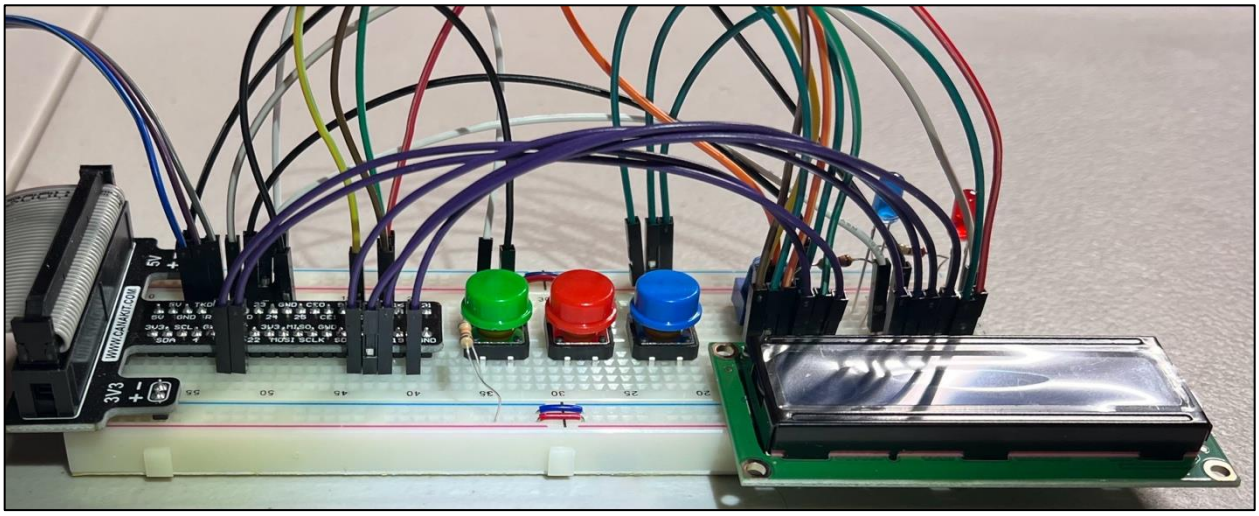
Note: The button will straddle the ditch between the rows of connected pins.

Once you have the button in place and the pins in the correct holes, apply firm pressure at each of the four corners of the button to seat it. You will know when the button has been seated when the base of the button is in contact with the solderless breadboard, and it does not rock when you press on any side.

You will now connect the second and final buttons, as installing both in sequence is easier before adding the additional connecting wires. Position the button so that the pins on one side of the switch are in Column D, Rows 21 and 23, while the other is connected to Column G, Rows 21 and 23.

Note: The button will straddle the ditch between the rows of connected pins.

Once you have the button in place and the pins in the correct holes, apply firm pressure at each of the four corners of the button to seat it. You will know when the button has been seated when the base of the button is in contact with the solderless breadboard, and it does not rock when you press on any side.

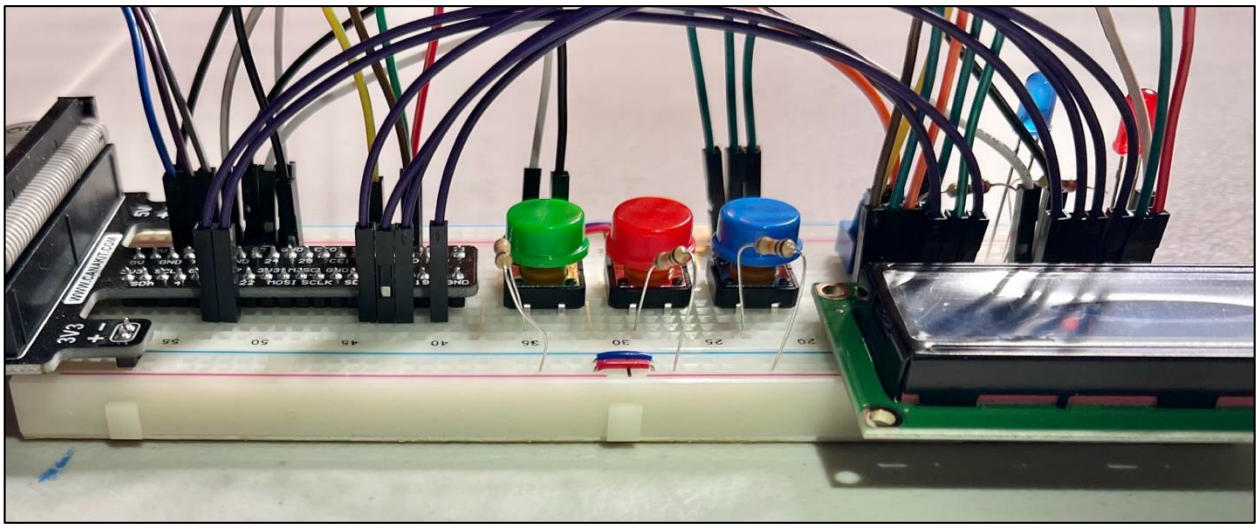


The next component to be placed is a 10K ohm resistor (stripe pattern: brown, black, orange). One leg of the resistor will need to be inserted into the +3.3v power rail at Row 2 while gently bending the legs so that the other leg can be inserted in Column B, Row 29.

Note: These are not in the same row due to the positioning of the switches and connecting wires on the power rail. This is not a problem.

Next, you will place the other 10K ohm resistor (stripe pattern: brown, black, orange). One leg of the resistor will need to be inserted into the +3.3v power rail at Row 22 while gently bending the legs so that the other leg can be inserted in Column B, Row 23.

Note: These are not in the same row due to the switches' positioning and the spacing on the power rail. This is not a problem.



Finally, you will install four jumper wires to connect the switches to the appropriate GPIO lanes.

The first jumper wire will connect the second half of the switch to ground. This is necessary so that when the button is pressed, the circuit is completed, and electricity flows from power through the resistor to ground.

Choose one jumper wire and connect Column H, Row 21 (again, using the numbering from the left-hand side of the board as viewed when the GPIO breakout board looks like a capital T) to Column I, Row 11 (numbers on the right-hand side of the board) (this should be the 10th pin on the right-hand side of the GPIO breakout board, labeled “gnd”).

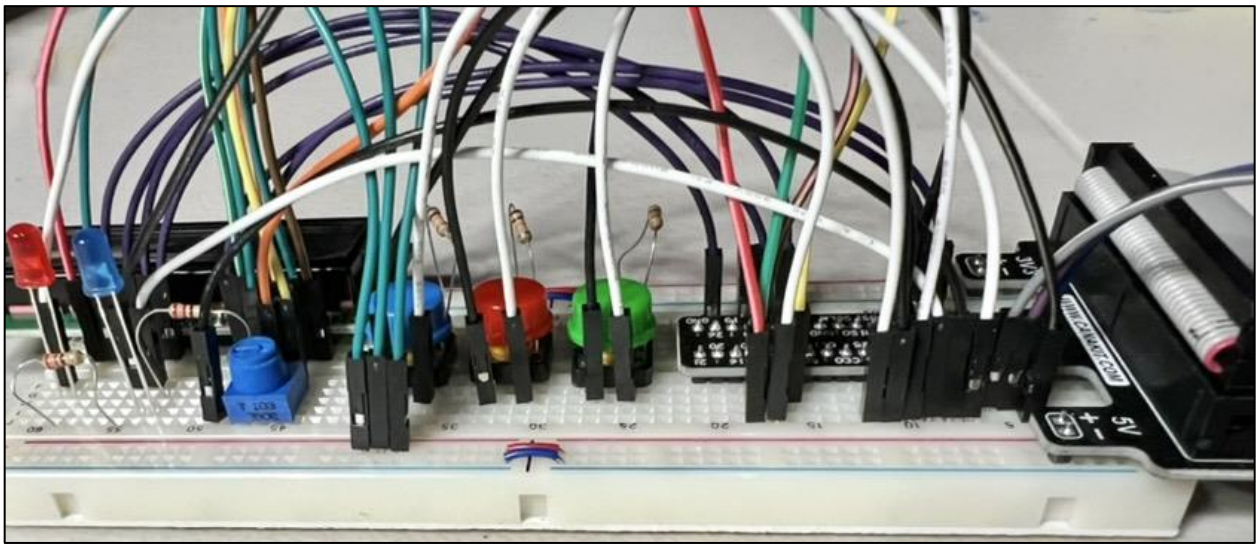
Choose one jumper wire and connect Column H, Row 27 (again, using the numbering from the left-hand side of the board as viewed when the GPIO breakout board looks like a capital T) to Column J, Row 11 (numbers on the right-hand side of the board) (this should be the 10th pin on the right-hand side of the GPIO breakout board, labeled “gnd”).

With the ground connections installed, the final jumper wires will connect the switches (on the same side as the resistor connection) to individual GPIO signaling pins.

Choose one jumper wire and connect it to Column H, Row 23 (the same row as one of the pins on the switch) and Column J, Row 12 (this should be the 11th pin on the right-hand side, labeled 25 on the breakout board).

Choose one jumper wire and connect it to Column H, Row 29 (the same row as one of the pins on the switch) and Column J, Row 17 (this should be the 16th pin on the right-hand side, labeled 12 on the breakout board).

At this point, you have completed the final modifications to the circuit, and your solderless breadboard should look like the image below:



Reconnect the power to your Raspberry Pi, and log in.

Step 3: Testing Your Circuit
Now that the full circuit is constructed, you will need to test all three buttons. The test script MultiButtonTest.py allows you to test all three buttons using the two LEDs in your circuit as indicators.

When you press the first button (the green one in the above picture), both LEDs will turn on solid.

When you press the second button (the red one in the above picture), the blue LED will turn off, and the red LED will fade in and out.

When you press the blue button, the red LED will turn off, and the blue LED will fade in and out.

You can run the script with the following command:

```
sudo python3 MultiButtonTest.py
```

Once you have verified that your circuit is functional, you are ready to complete the final project.

Step 4: Final Project Submission Requirements
For the final project, use the Thermostat.py script template provided in the CS 350 Code ZIP folder to complete the actions needed to satisfy the project's coding requirements.

Coding Requirements

1. The default set point should be 72 degrees Fahrenheit.
2. Use the AHT20 temperature sensor to read the room temperature (via I2C).
3. Use the two LEDs (red and blue) to indicate whether the system is heating or cooling.
    A. If the system is heating and the current temperature is below the set point, the red LED should fade in and out.
    B. If the system is cooling and the current temperature is above the set point, the blue LED should fade in and out.

   C. If the system is heating and the current temperature is at or above the set point, the red LED should be on and solid.
   D. If the system is cooling and the current temperature is at or below the set point, the blue LED should be on and solid.
4. The three buttons in the circuit will be used to change the behavior of the overall system.
   A. The first button is used to toggle the state machine between off, heating, and cooling.
   B. The second button is used to raise the system's set point by one degree.
   C. The third button is used to lower the system's set point by one degree.
5. The LCD in the circuit will be used to display the state of the thermostat to the user.
   A. The first line of the LCD should always be the date and current time of the system.
   B. The second line of the LCD should alternate between the current temperature and a line indicating which state the system is currently in and the temperature set point.
6. The UART must be configured to output the state of the thermostat over the serial port.
   A. The output should be a single string that is updated every 30 seconds.
   B. The output string should be comma delimited.
   C. The fields of the output string should be the following:
      i. State of the thermostat (off, heat, cool)
      ii. The current temperature
      iii. The temperature set point

Report Requirements

The global smart thermostat market is forecasted to reach almost $9 billion by 2026. SysTec's CEO wants to enter this lucrative market and has tasked your engineering department with creating a smart thermostat. SysTec develops analytics software for servers, but the CEO remembers from your interview that you took an embedded systems course and asks you to build a prototype.

The final goal is to develop a thermostat that sends data to SysTec's server software over Wi-Fi, but they first need a working prototype of the low-level thermostat functionality. You will create a written report for your team to ensure that the system you created is based on SysTec's business requirements and technical specifications.

While your team is testing your code, you have been asked to architect the project's next phase in your report: connecting the thermostat to the cloud.

Although you prototyped on the development board, the production product will utilize an integrated Wi-Fi solution. In this next phase, you are going to analyze various hardware architectures (from Raspberry Pi, Microchip, and Freescale) and recommend and justify the architecture decision based on the following business requirements:

- The thermostat must support the peripherals used in the project.
- The thermostat must connect to the cloud via Wi-Fi.
- The architecture chosen must have enough Flash and RAM to support the code.

In your report, you will compare the three hardware architectures based on the above requirements.